

Ischia Transfer Service

Audit Completo e Spietato

Valutazione del codice, dei moduli operativi, dei rischi di esercizio e della reale prontezza produttiva del progetto alla data del 26/04/2026.

Repository: c:\Users\user\Downloads\its

Stack: Next.js + TypeScript + Supabase + Vercel

Modalita: lettura diretta del codice e dei test locali

SEZIONE 1 — EXECUTIVE SUMMARY

Il progetto non e messo male: ha una copertura funzionale ampia, una base tecnica sensata, molte migration e diversi moduli operativi reali. Il problema e che l'affidabilita non e uniforme: alcune parti sono mature, altre sembrano complete ma non garantiscono davvero i vincoli chiesti dal cliente.

Funzionano bene il perimetro generale del prodotto, il modulo QR, buona parte del Piano del Giorno, gli export Excel e il modulo Medmar A/R come struttura. Il codice dimostra lavoro serio, velocita di iterazione e una base che si puo ancora salvare senza rifare tutto.

Quello che rischia di esplodere in produzione e molto concreto: regole dure che restano avvisi UI, auto-assign con fallback che puo violare i vincoli, editing manuale dei giri troppo permissivo, incoerenza tra lookup hardcoded e dati DB, impegni automezzo non integrati nella pianificazione, modulo Medmar A/R forte in facciata ma non ancora chiuso sui job automatici critici.

Stimerei 4-6 settimane uomo per arrivare a un prodotto realmente vendibile e difendibile in esercizio, a patto di fermare il cambio continuo di scope e chiudere prima i buchi server-side piu pericolosi.

SEZIONE 2 — VOTO FINALE

FUNZIONALITA

6.80/10

QUALITA CODICE

5.70/10

ARCHITETTURA

5.90/10

SICUREZZA

6.20/10

TEST E AFFIDABILITA

4.90/10

UX

7.10/10

VOTO FINALE PESATO

5.96/10

Il voto non e basso perche manca tutto. E basso perche il progetto e gia abbastanza grande da fare danni veri se le parti incomplete sembrano complete. La breadth e sopra la media; l'affidabilita dei vincoli operativi, delle integrazioni e dei calcoli economici non e ancora da produzione tranquilla.

SEZIONE 3 — STATO FUNZIONALITA

Legenda:  **COMPLETATA E TESTATA**  **IMPLEMENTATA MA NON TESTATA / CON BUG MINORI**
 **IMPLEMENTATA PARZIALMENTE**  **NON IMPLEMENTATA**  **INFORMAZIONI INSUFFICIENTI**

MODULO	FUNZIONALITA	STATO	COSA MANCA	EFFORT RESIDUO
Gestione Prenotazioni	Ricerca nome/cognome/telefono		La route <code>app/api/ops/search/route.ts</code> cerca <code>customer_name</code> e <code>phone</code> , non <code>nome/cognome</code> separati.	0.5g
Gestione Prenotazioni	Inserimento manuale prenotazioni		Funziona, ma con workaround di schema drift in <code>app/api/ops/new-booking/route.ts</code> e senza copertura end-to-end robusta.	1-2g
Gestione Prenotazioni	Campo numero mezzo condizionale		Esiste come <code>transport_code</code> contestuale, non come etichetta esplicita uniforme.	0.5g
Gestione Prenotazioni	Logica porto SNAV sempre Casamicciola		Presente ma hardcoded in <code>lib/snav-medmar-lookup.ts</code> .	0.5g
Gestione Prenotazioni	Logica porto Medmar per orario		Presente ma hardcoded e non centralizzata sul DB.	1g
Gestione Prenotazioni	Fix sostituisci duplicato 404		Route presente e test di integrazione dedicato in <code>tests/integration/service-replace.test.ts</code> .	0g
Gestione Prenotazioni	Rimozione SNAV 08:10		La migration c'e, ma il codice applicativo e il seed di stress contengono ancora <code>08:10</code> .	0.5-1g
Biglietteria	Invio Multiplo Agenzie		La pagina esiste; da correggere la lista target: in codice c'e <code>Ischia Stars</code> al posto di <code>Sosandra</code> .	0.5g
Biglietteria	Upload fino a 10 PDF		Limite client+server presente, ma senza test automatici mirati.	0.5g
Biglietteria	Invio massivo con email		Implementato con Resend in <code>app/api/biglietti/send-bulk/route.ts</code> , non testato.	1g
Escursioni	Capienza automatica e posti realtime		Buona UI in <code>app/(app)/escursioni/page.tsx</code> , ma enforcement server debole.	1g
Escursioni	Avviso overbooking		Presente lato UI, non bloccante lato API.	0.5-1g
Automezzi	Impegni automezzo		Tabelle e UI presenti, ma integrazione operativa incompleta.	0.5g

MODULO	FUNZIONALITA	STATO	COSA MANCA	EFFORT RESIDUO
Automezzi	Blocco disponibilita da impegno		vehicle_commitments non risultano usati dal motore disponibilita/piano.	2-3g
QR Smarcamento	QR, vista mobile, conferma, blocco doppio, log		Modulo concreto e ben strutturato, ma con poca copertura automatica oltre allo smoke/E2E principale.	1g
Piano del Giorno	Schermata completa con pool, creazione giri, timeline autisti		UI ricca e viva in app/ (app) /piano-giorno/page.tsx.	0.5g
Piano del Giorno	Validazioni conflitti e capienza		Sono soprattutto segnali UI; la route manuale app/api/ops/piano-giorno/trips/route.ts non blocca davvero.	2-4g
Piano del Giorno	Realtime da smarcamenti QR		Subscription presente su services, assignments, trip_groups, status_events.	0.5g
Piano del Giorno	Stampa piani autista		Stampa HTML client-side buona, non PDF server-side nativo.	1g
Semi-Automatica	Genera proposta automatica		Funzione presente e integration test parziale su auto-assign.	0.5g
Semi-Automatica	Clustering geografico ARRIVI		La logica nordovest/estsud c'e in auto-assign/route.ts, ma non e testata in modo esaustivo.	1-2g
Semi-Automatica	Vincoli rigidi mai violabili		Falso nello stato attuale: il fallback veicolo puo aggirare i limiti e il fallback driver puo accettare conflitti.	3-5g
Semi-Automatica	Report non assegnati con motivazioni		Il report e troppo aggregato, non per singolo servizio con causa precisa.	1-2g
Disponibilita	Schermata giorno prima con conferma		Buona base e gate server attivo sull'auto-assign.	0.5g
Disponibilita	Conferma obbligatoria prima del Piano del Giorno		Blocco server su auto-assign si, blocco totale del workflow piano no.	1-2g
Hotel ↔ Mezzo	Tabella limiti popolata		Seed presente in 0142_hotel_vehicle_limits_seed.sql.	0.5g
Hotel ↔ Mezzo	Validazione bloccante non aggirabile		Non e garantita su tutte le API; il manuale non la applica e l'auto-assign ha fallback permissivi.	2-3g
Autista ↔ Mezzo	Campo e seed limiti presenti		Migration 0143_driver_max_vehicle_capacity.sql	0.5g

MODULO	FUNZIONALITA	STATO	COSA MANCA	EFFORT RESIDUO
			corretta.	
Autista ↔ Mezzo	Validazione bloccante		Stesso problema dei limiti hotel: non e centralizzata e non e non-aggirabile.	2-3g
Output	Fase A assegnazione / Fase B file		Export bloccato se mancano giri/assegnazioni, buona idea ma senza garanzia completa lato backend.	0.5g
Output	Excel multi-foglio		Presente e sostanzioso in <code>app/api/ops/piano-giorno/export-excel/route.ts</code> .	0.5g
Lista Bruno	Criteri inclusione richiesti		La logica in <code>lib/server/continent-dispatch.ts</code> e parziale rispetto alle regole cliente piu specifiche.	2-3g
Lista Bruno	Dati completi e numero volo corretto		Meglio di altre aree: route ed export gestiscono arrivo/partenza con campi separati.	0.5g
Medmar A/R	Tabelle separate + helper + dashboard		Modulo ampio e reale, ma non ancora chiuso sul piano operativo notturno e sulle notifiche.	1-2g
Medmar A/R	Decision Helper 3 scenari		Refactoring e test dedicati presenti.	0g
Medmar A/R	Job batch scadenza notturna		Nessuna route cron o voce in <code>vercel.json</code> chiaramente dedicata al processo.	2-3g
Medmar A/R	Notifiche scadenze 24h		Non trovate nel codice.	1-2g
Medmar A/R	Matching automatico		C'e, ma e un matching semplice per tratta/data/orario, non un motore particolarmente robusto.	2-3g
Stress Test	Dataset 500 servizi + cleanup		Script reale in <code>scripts/seed-test-sunday.mjs</code> , ma senza certificazione CI/E2E del flusso completo.	1-2g

SEZIONE 4 — PUNTI VERAMENTE CRITICI

1. BLOCCANTE — I vincoli rigidi non sono davvero rigidi.

Le regole hotel ↔ mezzo e autista ↔ mezzo non sono garantite in tutte le API. L'auto-assign tenta di rispettarle, ma ha fallback permissivi. Il flusso manuale dei giri in `app/api/ops/piano-giorno/trips/route.ts` e sostanzialmente cieco a questi vincoli.

2. BLOCCANTE — Gli impegni automezzo esistono ma non governano davvero la pianificazione.

`vehicle_commitments` sono visibili da fleet ops, ma non risultano integrati nella disponibilit  usata da Piano del Giorno e auto-assign. Operativamente   una bomba a orologeria.

3. ALTA — Drift tra migration e codice su orari SNAV/Medmar.

Hai pulito il DB sulla corsa 08:10, ma il codice hardcoded e lo stress seed non sono allineati. Questo   esattamente il tipo di drift che produce prenotazioni giuste nel DB e sbagliate nella UX.

4. ALTA — Il Piano del Giorno segnala problemi che il backend non impedisce.

La UI   piena di blocker e warning utili, ma senza equivalenti 409 server-side quei blocker sono cosmetica operativa.

5. ALTA — Modulo Medmar A/R forte in facciata, meno forte nella chiusura batch.

Helper, KPI, export e simulatore ci sono. Manca la prova di una catena notturna robusta su scadenze e notifiche 24h. Per un modulo economico, questo   troppo pericoloso.

6. ALTA — Type-safety Supabase ancora incompleta.

`lib/supabase/types.ts` non riflette ancora davvero il perimetro delle tabelle operative presenti nelle migration. Hai aggiunto il check CI, ma il file attuale resta incompleto.

SEZIONE 5 — DEBITO TECNICO ACCUMULATO

- **Lookup traghetti hardcoded.** Debito: il DB evolve e il codice no. Esplode al primo cambio orario. Effort: 2-3g.
- **Schema drift tollerato nel codice.** Debito: fallback applicativi in booking, fleet ed escursioni al posto di ambienti allineati. Esplode ai deploy non perfetti. Effort: 3-5g.
- **Copertura test sbilanciata.** Debito: Vitest misura soprattutto helper in `lib/`, non le route operative in `app/`. Effort: 4-7g.
- **Doppia sorgente sui mezzi.** Debito: moduli diversi usano `vehicles` e `vehicle_records`. Effort: 2-3g.
- **Ruoli non armonizzati.** Debito: `lib/rbac.ts` e `lib/validation.ts` non parlano esattamente la stessa lingua. Effort: 1-2g.

SEZIONE 6 — ROADMAP RACCOMANDATA AL COMPLETAMENTO

FASE 1 — STABILIZZAZIONE

- Centralizzare e bloccare lato server i vincoli hotel ↔ mezzo, autista ↔ mezzo, conflitti minimi e disponibilit .
- Integrare `vehicle_commitments` nel motore di disponibilit .
- Rimuovere il drift SNAV/Medmar tra codice e DB.
- Rigenerare i tipi Supabase e chiudere le incoerenze di ruoli.

Effort stimato: 1.5-2.5 settimane

FASE 2 — COMPLETAMENTO FUNZIONALITA

- Chiudere Bruno sui criteri esatti richiesti e sugli output.
- Correggere il modulo Invio Multiplo Agenzie.
- Chiudere Medmar A/R su batch scadenze, notifiche 24h e matching piu robusto.
- Rendere davvero obbligatoria la conferma disponibilit  prima del workflow piano.

Effort stimato: 2-3 settimane

FASE 3 — OTTIMIZZAZIONE E HARDENING

- Test integration seri sulle route operative del piano, disponibilit , export, Bruno e Medmar.
- Stress test eseguibile in modo ripetibile e certificabile.
- Restore test dei backup, monitoring minimo e politica di rollback/staging.

Effort stimato: 1.5-2 settimane

SEZIONE 7 — RISCHI BUSINESS

- **Rischio operativo:** piani generati o modificati che non sono applicabili davvero la mattina successiva.
- **Rischio economico:** dashboard e simulatore Medmar possono guidare decisioni con catena operativa di recupero ancora incompleta.
- **Rischio reputazionale:** porto o orario traghetto incoerente tra codice e DB significa cliente nel posto sbagliato.

- **Rischio commerciale:** il prodotto in demo appare più maturo di quanto sia nei punti critici di backend.
- **Rischio legale/privacy:** non vedo una falla clamorosa nel repo, ma monitoring, restore testato e audit autorizzativo completo sono ancora deboli.

SEZIONE 8 — RACCOMANDAZIONE FINALE

NO ASSOLUTAMENTE.

Non perché il progetto sia scarso, ma perché oggi promette più controllo operativo di quello che il backend garantisce davvero. Andare live così significa affidare una domenica ad alto carico a un software che in alcune aree è convincente in UI ma ancora permissivo nei punti dove dovrebbe dire no.

Documento generato da audit locale del repository. I punti classificati come insufficienti o parziali indicano aree non verificabili in modo definitivo dal solo codice presente.